

Końcowy raport - rekonstrukcja

Metodologia

Wstęp

Ostatnio zmniejszyliśmy gęstość binnigu. Umożliwia to sprawdzenie także architektury MLP (Multi Layer Perceptron ...), która w teorii uwzględnia globalne cechy

Architektury

Architektury który sprawdziliśmy : - jednowarstwową architekturę MLP (wymiar: 512) - dwuwarstwową architekturę MLP (wymiar: 512 x 256) - sieć konwolucyjną wymiary: - channels: [1, 32, 16, 8] - kernels: [7, 5, 3] - strides: [3, 3, 3]

Dla wszystkich sieci rozmiar latentu wynosił 10.

Binning

Binning który sprawdziłem: - 0.45 - 0.5 - 0.55 - 1.

Pierwsze trzy miały sprawdzić czy dodatkowe rozróżnienie (jeżeli tak to jak gęste), pomoże w uczeniu modelu.

Ostatni jest **najmniejszym** rozmiarem binu jaki możemy zastosować.

Uwaga: Ważne jest tutaj założenie, że technologia MALDI ma rozdzielność 1 m/z, przesunięcia w osi wynikają z drobnych różnic mas molekularnych, oraz stanu fizykochemicznego jonu.

Dane

Zrobiłem losowy dobór pixeli z 10 % próbki ($\approx$ 3gb). Tkanką była nerka. Uwaga, na tym etapie **nie normalizowałem** danych ze względu na analizator, nie ma to tutaj znaczenia, ponieważ on wpływa tylko na położenie cząsteczek na widmie - będzie to miało wpływ podczas annotacji.

Wszystkie kombinacje modeli były uruchomione 5 razy.

Wyniki

Ogólnie

Rekonstrukcja widma

Otrzymujemy dokładność na poziomie $\pm 1\text{Da}$, plus ewentualnie szum.

Wyszło to trochę gorzej niż ostatnio (dopasowanie widm). Wynika to z tego że przestrzeń jest mniejsza, mamy mniej złożone modele oraz, że błąd był sztucznie **zaniżony**. Uśredniałem go po batchach itd. a przez to że wymiar był duży to wyniki wychodziły małe. Nie robiłem dokładniejszej analizy ale też pewin były mismatche o kilka pixeli.

Najlepszy model

Najlepszym modelem sie okazuje siec konwolucyjna **ale** dopiero na binnigu 0.55, 1.0, poniewaz wtedy zaczyna generowac obwiednie.

Wstępna analiza (wiem - nic to nie mówi)

Złożoność

Siec konwolucyjna ma 100 razy mniejszą złożoność. Jej wadą jest to że nie wychwytuje globalnej informacji.

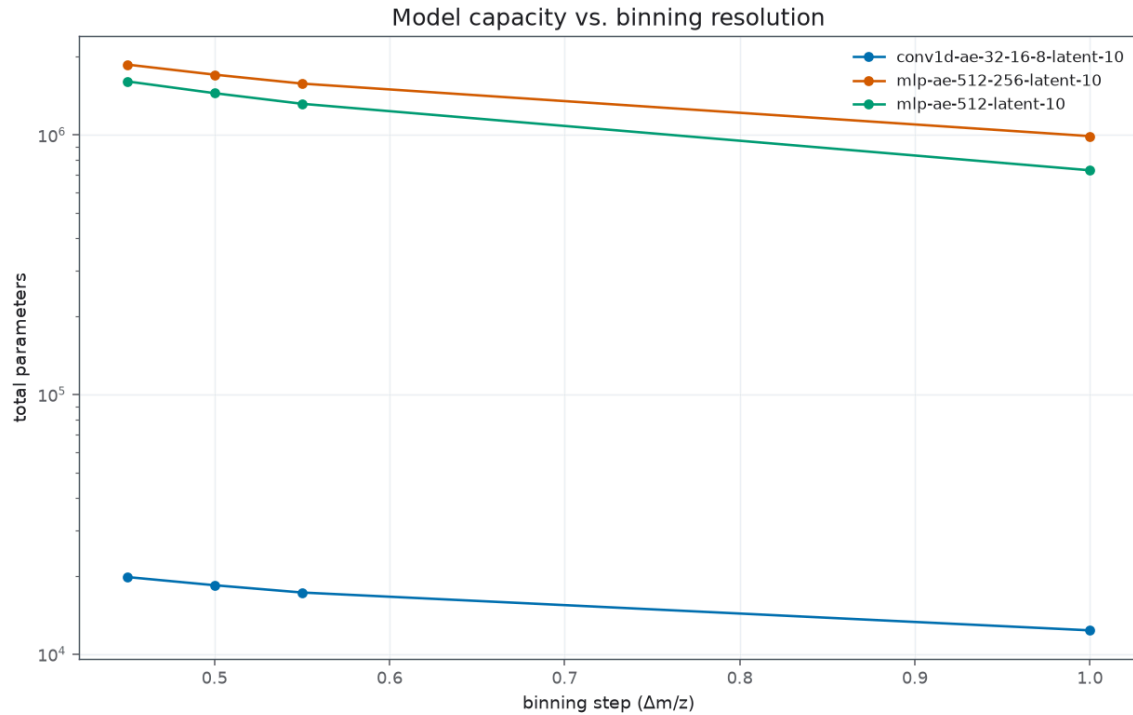


Figure 1: model_capacity_vs_binning_resolution

Tabela ze złożonością

architecture	binning_step	total_parameters	mean_epoch_duration	std_epoch_duration	mean_total_duration	epoch_count
conv1d-ae-32-16-8-latent-10	0.45	19825	69.850553	6.209009	698.505534	50
conv1d-ae-32-16-8-latent-10	0.50	18433	65.247411	2.326418	652.474106	50
conv1d-ae-32-16-8-latent-10	0.55	17279	64.538101	1.094870	645.381005	50
conv1d-ae-32-16-8-latent-10	1.00	12353	62.307517	0.641353	623.075168	50
mlp-ae-512-256-latent-10	0.45	1866782	66.990728	8.425594	669.907284	50
mlp-ae-512-256-latent-10	0.50	1706882	73.831983	6.734934	738.319826	50
mlp-ae-512-256-latent-10	0.55	1576707	77.399525	10.925777	773.995251	50
mlp-ae-512-256-latent-10	1.00	989382	68.265170	7.547060	682.651697	50
mlp-ae-512-latent-10	0.45	1608222	60.335828	4.232329	603.358281	50
mlp-ae-512-latent-10	0.50	1448322	61.606467	1.886089		

616.064668 50 10 mlp-ae-512-latent-10 0.55 1318147 62.066333 1.133055 620.663334 50 11 mlp-ae-512-latent-10 1.00 730822 64.916190 8.322276 649.161901 50

Czas trenowania

Widzimy, że złożoność modelu, **nie wpływa** na długość treningu.

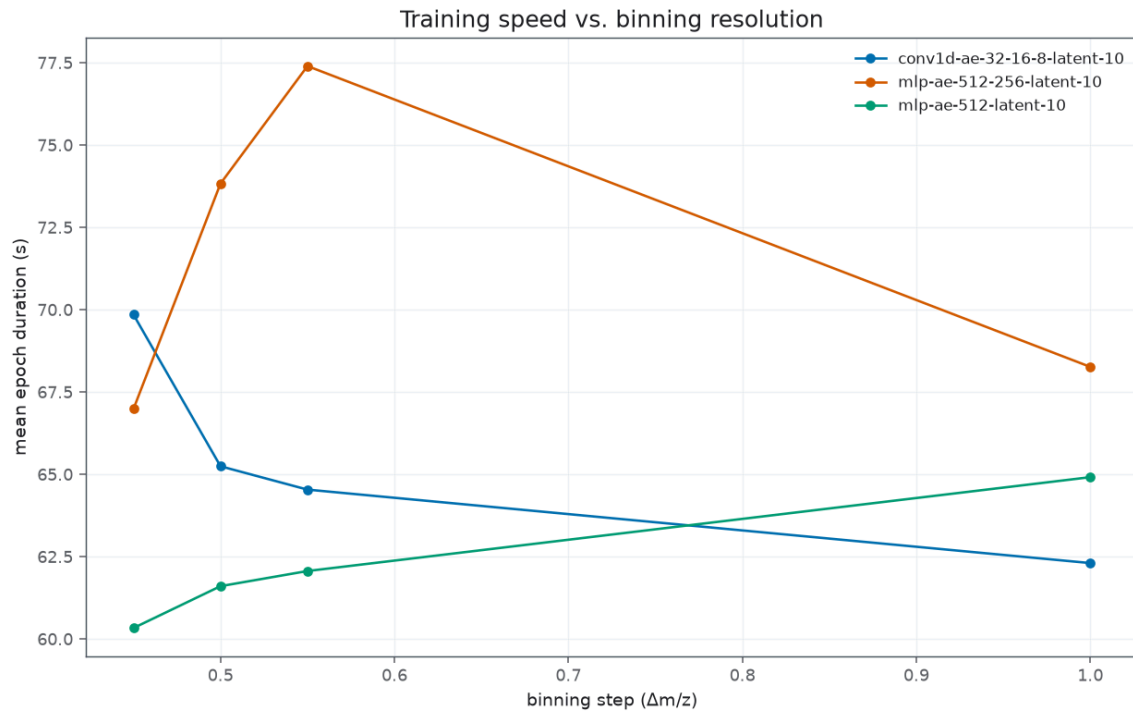


Figure 2: training_speed_cs_binning_resolution

Porównanie ogólnej funkcji kosztu

Podczas trenowania zapisywałem modele, które były najlepsze względem **zbioru walidacyjnego**. Na każdym wykresie przyjmujemy: - ciągła linia: treningowy zbiór - kreskowana linia: zbiór walidacyjny

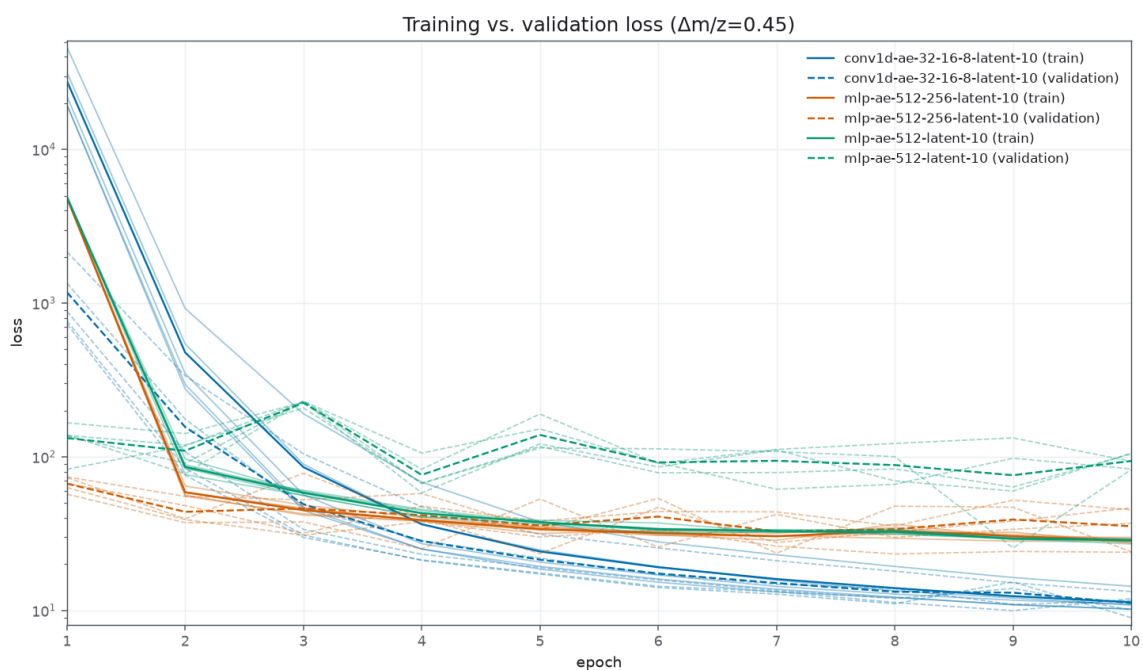


Figure 3: train_vs_validation_0_45.png

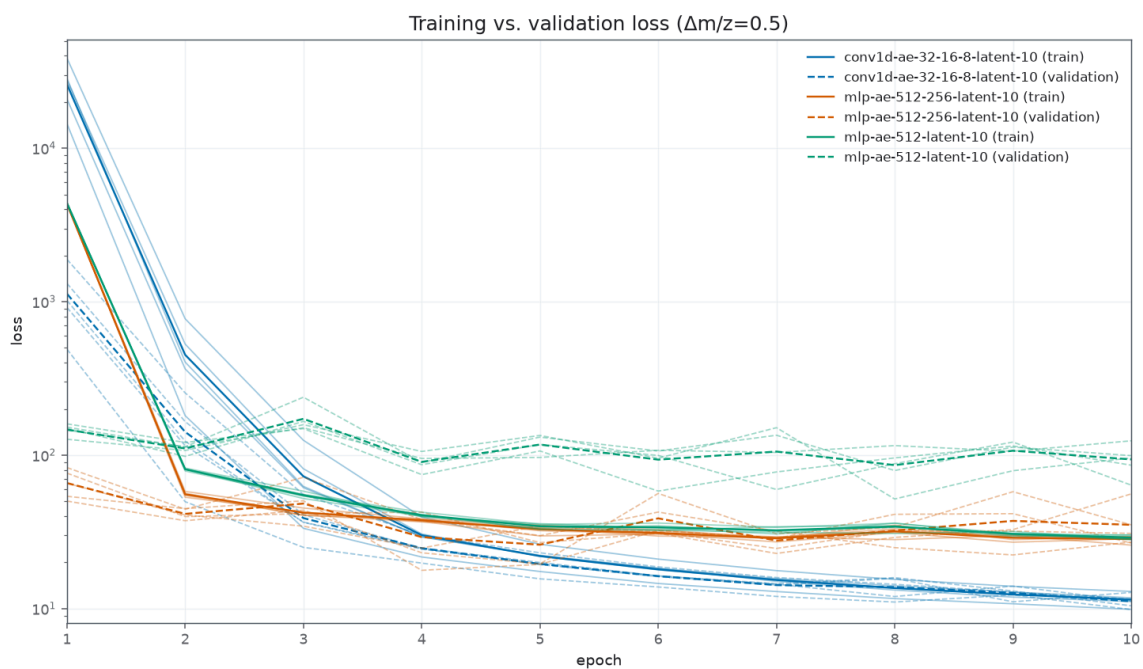


Figure 4: train_vs_validation_0_50.png

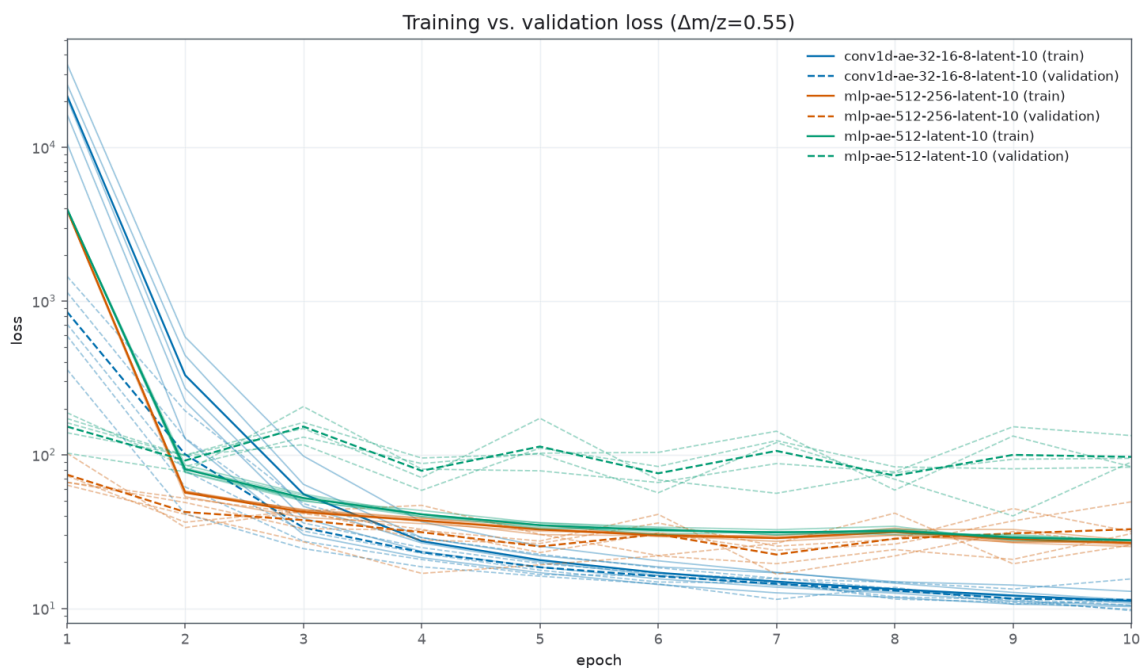


Figure 5: train_vs_validation_0_55.png

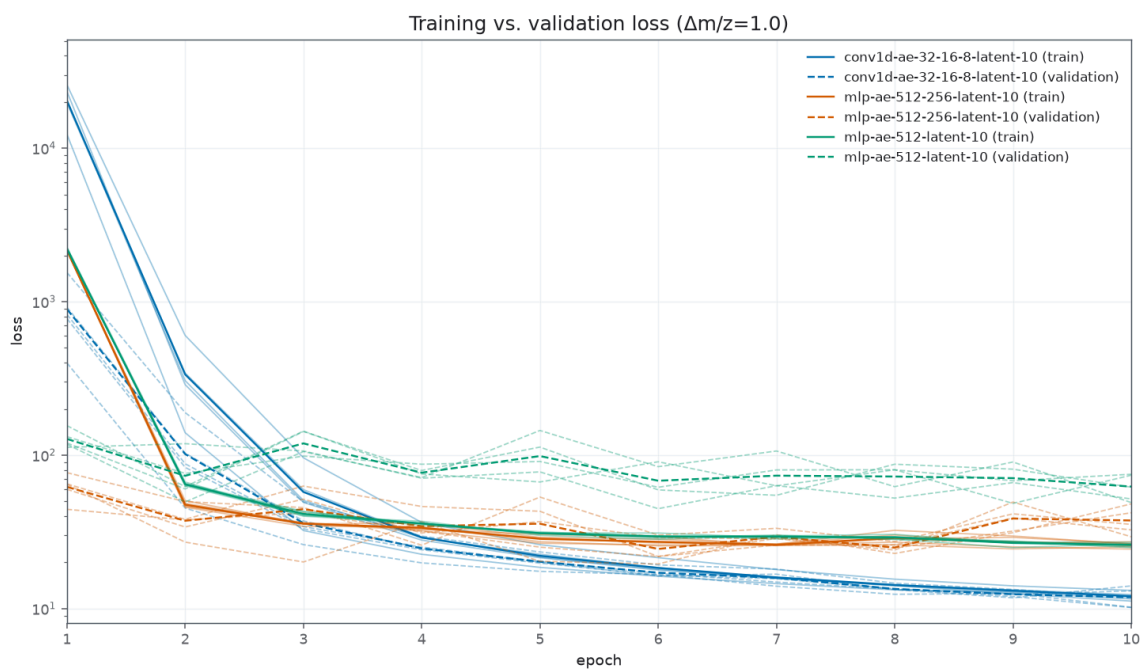


Figure 6: train_vs_validation_1_00.png

Overfitting

Widzimy, że model **MLP-512** bardzo szybko overfittuje, **MLP-512-256** nie overfittuje ale ma większy błąd, najlepiej dopasuje się **conv**.

Stabilność

Tutaj było dla mnie ważne, żeby zobaczyć jak binning wpływa na wariancję wartości funkcji straty. Zauważmy że w przypadku binningu $\Delta m \setminus z = 0.55$, $\Delta m \setminus z = 1.0$

Analiza rekonstrukcji widm

Rozkład błędu

Widzimy, że ogólnie błąd dla $\Delta m \setminus z = 0.55$, $\Delta m \setminus z = 1.0$, rozkład błędu jest znacznie mniejszy.

Dla przypomnienia dolna wartość takiego błędu, idealnie dopasowane widmo, to jest

Konwolucyjna sieć

#TODO - nazwać odpowiednio obrazy

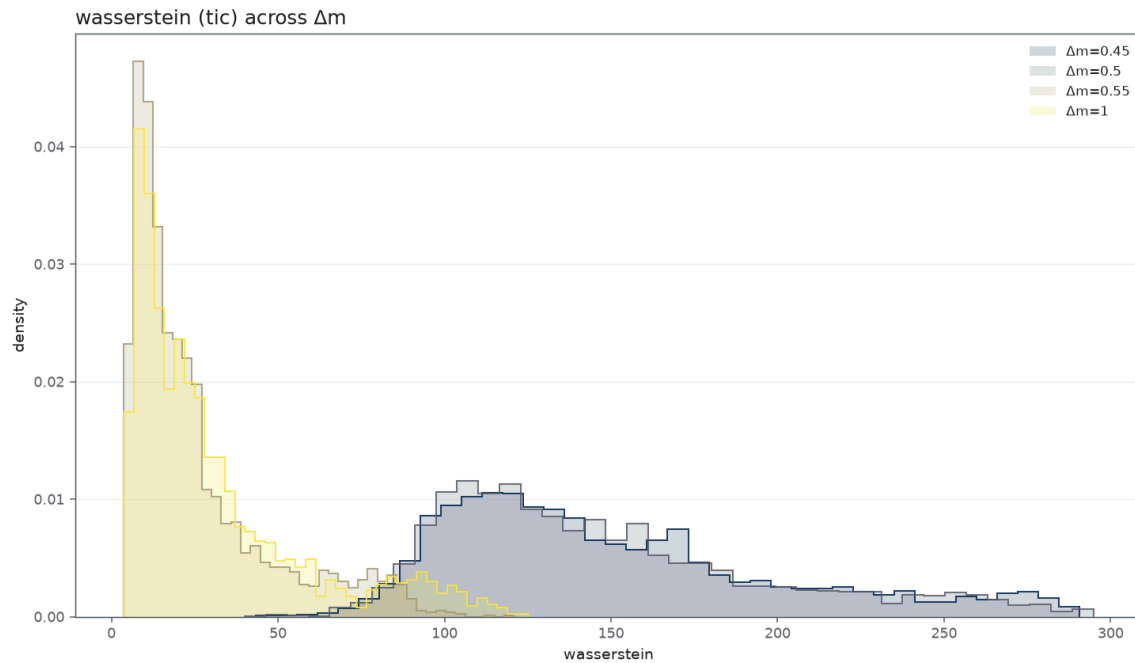


Figure 7: conv

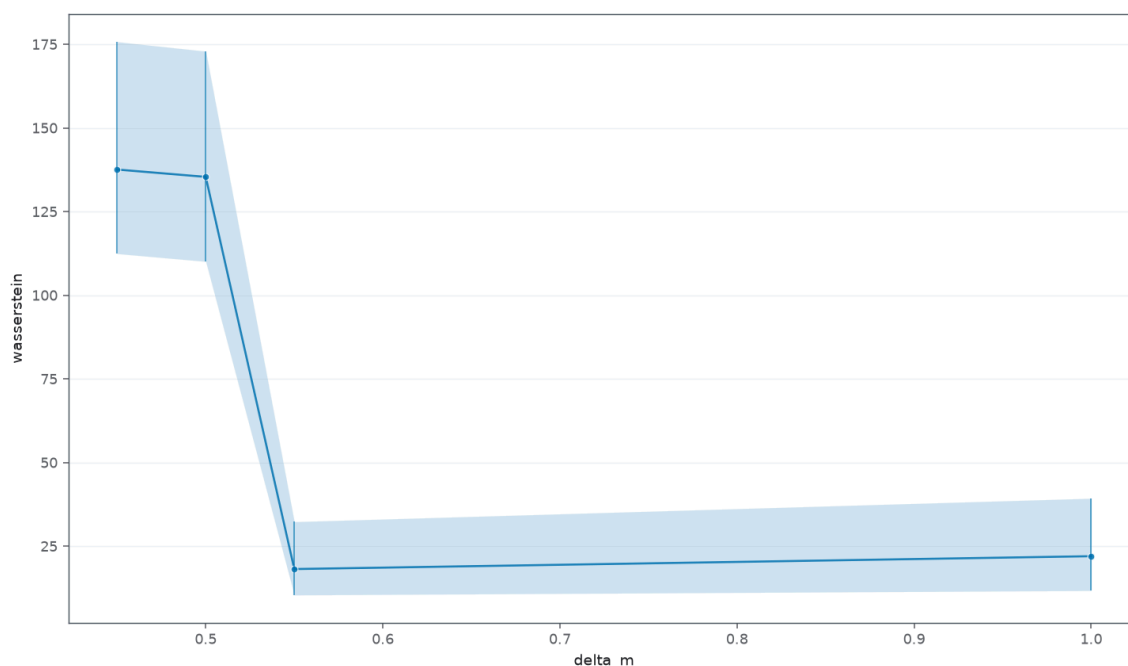


Figure 8: conv

Porównanie widm - sieć konwolucyjna

Tutaj nie zamieszczam zdjęć, ponieważ jest tego za dużo. Zamieszczę moje obserwacje, żeby sobie porównać zdjęcia, proszę zerknąć do notebooka (#TODO - github - link).

Różnica pomiędzy binning'em $\Delta m \setminus z = 0.50$, $\Delta m \setminus z = 0.55$

Różnica polega na tym że dla binningu $\Delta m \setminus z = 0.55$, sieć konwolucyjna **zaczyna** generować obwiednie.

Rozkład m/z

Nie wiem o co chodzi z peakiem na wartości 900. Mam tezę, że model **nie potrafił**, dopasować całej wartości. Żeby funkcje kosztu zmniejszyć potrzeba mieć **takie same intensywności**, jak tego nie ma to otrzymujemy bardzo dużo błąd (#TODO - policzyć taki przykład)

Problem znika dla binningu $\Delta m \setminus z = 0.55$, $\Delta m \setminus z = 1.0$, tutaj model umei ZNACZNIE lepiej dopasować te widma oraz intensywności, nie powstaje jeden ogólny “śmietnikowy” bin który wszystko zbiera.

Szum

Zauważmy, że w przypadku sieci konwolucyjnych występuje “szum” (obwiednia) wokół pików. Podejrzewam, że filtry które używamy wykrywają, że w danym otoczeniu latentu peak powinien występować dla pewnego $m \setminus z$ Nie jest to aż tak straszne, ponieważ symuluje obwiednie i ten błąd to jest kilka daltonów.

Warto zauważyć, że to wpływa na charakterystykę **najgorzej przewidzianych widm**, błąd jest tam największy ponieważ powstaje w regionach **rzadko rozłożonych peaków** - powstaje obwiednia w ic otoczeniu a nic tam nie powinno być.

Treningowe vs testowe

W treningowych widmach widać mniejsze obwiednie., ale nie ma wyraźnych różnic.

Uwaga: W notebooku najpierw są testowe potem treningowe zaplotowane

Podsumowanie

Błąd wynika głównie z **nadmiernego** generowania widm, wynika to z wielkości filtra (ograniczmy go do 3, żeby to otocznie zredukować), lokalizacja peaków jest dobra, błąd wynosi zazwyczaj około $\pm Da$

MLP

Komentuje tutaj wyniki obu architektur na raz

Dokładność intensywność

Ten model ma mniejszy problem z dopasowaniem intensywności, jednak

Ubogość widm

Tutaj głównym problemem jest to że generowane widma są zbyt rzadkie, powoduje to, że widzimy tylko pojedyncze peaki, są one z błędem większym niż $\pm Da$.

Podsumowanie

Ostateczny binning

Ostatecznie bym wybrał binning $\Delta m \setminus z = 0.55$. Jest on najstabilniejszy podczas po treningu. Jest to dobra informacja, ponieważ ja tą wartość interpretuje jako *najmniejszą wartość binningu, która pozwala modelowi na prawidłową rekonstrukcję (generowanie obwiedni), jednocześnie zachowując najwięcej wyjściowej informacji.*

Ostateczna architektura - sieć konwolucyjna

Zdecydowanie lepiej radzi sobie sieć konwolucyjna, ponieważ **nadmierną** ilość widm możemy wyciszyć, w przypadku MLP mamy za mało widm, nie możemy ich sztucznie wygenerować.

Propozycje poprawy

Kara do nierównomiernego rozłożenia wartości

Suma wyjściowa ma być 1 (bo normalizację TiC'iem), można wymusić, żeby nie było takiej sytuacji że dostaję skrajne wartości, więc można by dodać karę za nierównomiernie rozłożenie tej masy, żeby nie było tak że model wybiera sobie punkt w którym daje prawie całą masę, ponieważ jest to mniejszy błąd niż w przypadku dopasowania

Centroidowanie widma

Moglibyśmy zrobić coś takiego, że centoridujemy te rozrzucone widma jakoś (przy decoderze jakąś poprawkę zastosować) - pomysły na to.